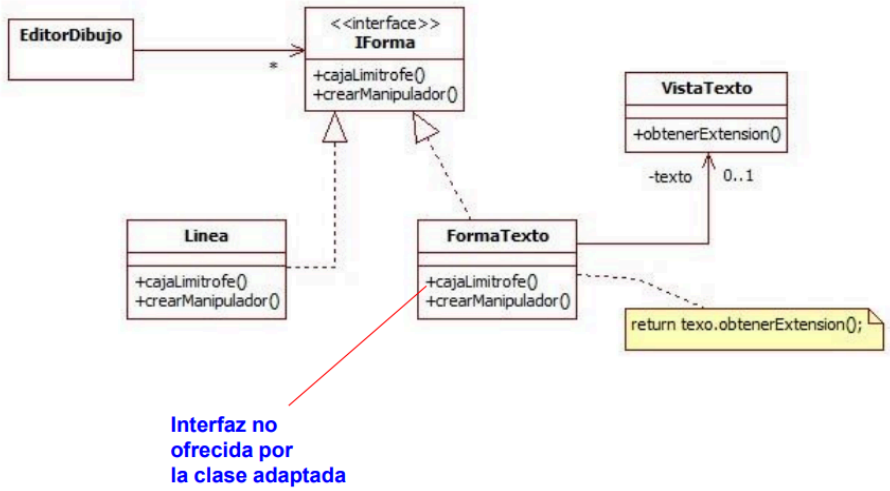
T9. Patrones estructurales

Adapter

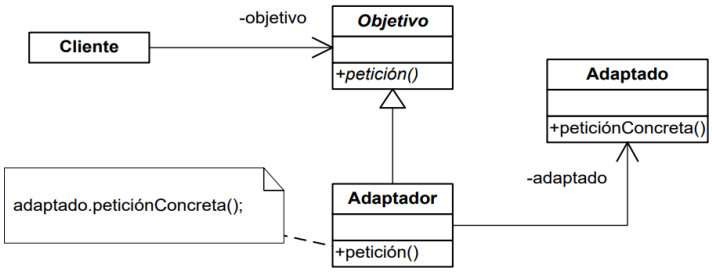
El patrón Adapter **convierte** la interfaz de una clase en otra que es la que realmente esperan sus clientes. Esto permite que clases con **interfaces incompatibles** puedan cooperar sin necesidad de modificar sus implementaciones originales. Se usa cuando: se quiere usar una clase existente cuya interfaz no concuerda con la interfaz necesaria que necesita el cliente; se quiere crear una clase reutilizable que coopere con clases no previstas originalmente; es necesario usar varias subclases existentes y no es práctico heredar para adaptar su interfaz.



A la izquierda está **EditorDibujo**, que es el cliente que interactúa con las interfaces adaptadas, esperando objetos que implementen la interfaz **IForma**. Esta interfaz define las operaciones cajaLimite() y crearManipulador(), que son requeridas por el cliente. En el centro del diagrama, tenemos **FormaTexto**, que es la clase que necesita ser adaptada. FormaTexto no cumple directamente con la interfaz IForma, pero su objetivo es proporcionar la funcionalidad esperada por el cliente. Para hacer esto, FormaTexto será **adaptada** mediante el patrón Adapter. La clase VistaTexto es un ejemplo de un Adaptador en este caso. El adaptador implementa la interfaz IForma y actúa como **intermediario** entre FormaTexto y el cliente. La clase VistaTexto adapta la interfaz de FormaTexto y proporciona los métodos esperados, como el método obtenerExtension().

- <u>Participantes</u>

- **Objetivo**: define *interfaz* específica del dominio que usa el Cliente.
- **Cliente**: colabora con objetos que se ajustan a la interfaz Objetivo.



- **Adaptado**: define la interfaz existente que necesita ser adaptada.
- **Adaptador**: adapta las interfaces Adaptado y Objetivo.

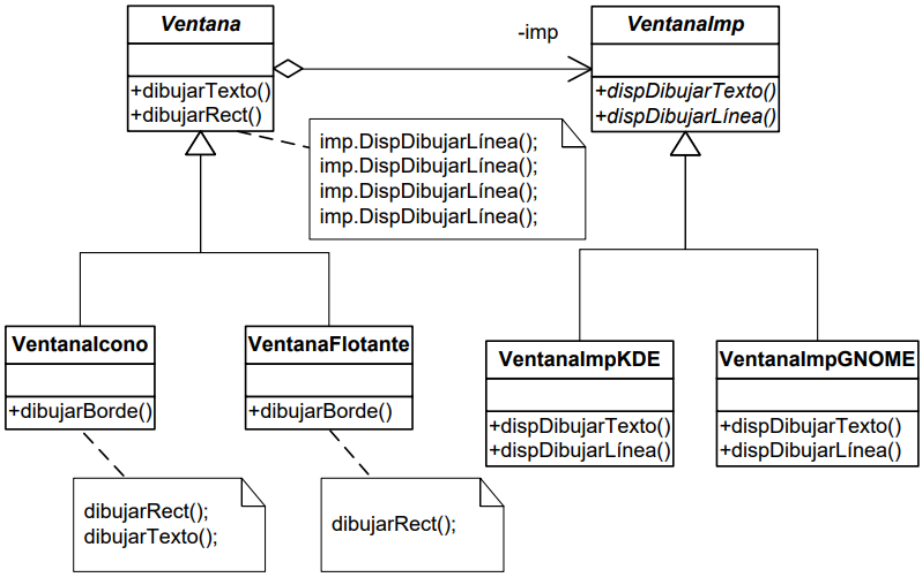
- <u>Ventajas e inconvenientes</u>

Permite al mismo Adaptador funcionar con muchos Adaptados y añadirles funcionalidad. Sin embargo, es más difícil redefinir comportamiento de Adaptado, crear subclases de Adaptado y hacer que Adaptador se refiera a ellas.

Bridge (Puente)

El patrón **Bridge** separa una **abstracción** de su **implementación**, permitiendo que ambas puedan variar de forma independiente. Se creó debido a la necesidad de varias implementaciones posibles para la misma abstracción, siendo el uso de herencia poco flexible. Se usa:

- Para evitar un vínculo permanente entre una abstracción y su implementación, especialmente si dicha decisión debe tomarse en tiempo de ejecución.
- Cuando necesitas que tanto la abstracción como la implementación puedan ser **ampliadas** mediante herencia.
- Para impedir que los cambios en la implementación **afecten** a los clientes, lo que evita recompilaciones al cambiar la implementación.
- Cuando necesitas **compartir** la **implementación** entre varios objetos, ocultándola de los clientes.

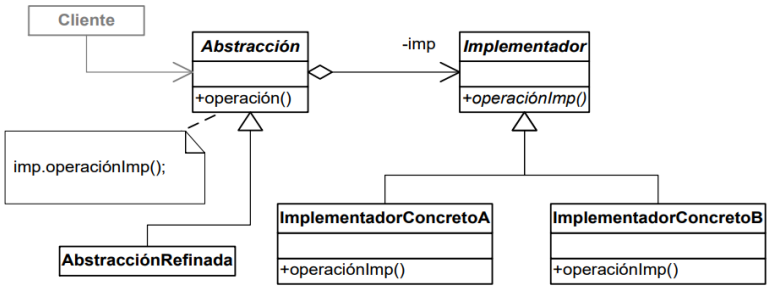


En la parte superior izquierda está la clase **Ventana**, que representa la abstracción. Esta clase define los métodos como dibujarTexto() y dibujarRect(), que son operaciones generales que todas las ventanas deben ser capaces de

ejecutar, independientemente de su implementación. Debajo de Ventana tenemos las clases **VentanaIcono** y **VentanaFlotante**, que son subclases de la abstracción que implementan operaciones específicas de la ventana, como dibujarBorde(). En el centro del diagrama está la clase **VentanaImp**, que es la **implementación abstracta** de la que se derivan las clases concretas como VentanaImpKDE y VentanaImpGNOME. La clase Ventana (la abstracción) no está directamente ligada a una implementación específica, tiene una referencia a VentanaImp (la interfaz de implementación), y las subclases de VentanaImp son las que implementan los detalles gráficos específicos.

- <u>Participantes</u>

- **Abstracción**: define interfaz de la abstracción y referencia a un objeto de tipo Implementador.
- **AbstracciónRefinada**: implementa la interfaz definida por Abstracción.
- **Implementador**: define interfaz para clases de implementación (habitual que sus operaciones sean de más bajo nivel que las de Abstracción).
- **ImplementadorConcreto**: implementa la interfaz Implementador.



- <u>Ventajas e inconvenientes</u>

Desacopla la **interfaz** de la implementación, permitiendo que la implementación de una abstracción sea configurada en tiempo de ejecución y pudiendo eliminar dependencias de compilación. Además, permite que la Abstracción y el Implementador se desarrollen y mantengan independientemente, permitiendo ocultar detalles de implementación a los clientes.

- <u>Adapter vs Bridge</u>

- **Adapter** resuelve incompatibilidades entre interfaces existentes sin preocuparse por cómo podrían evolucionar independientemente. Su propósito es **adaptar interfaces incompatibles** para que puedan trabajar juntas.

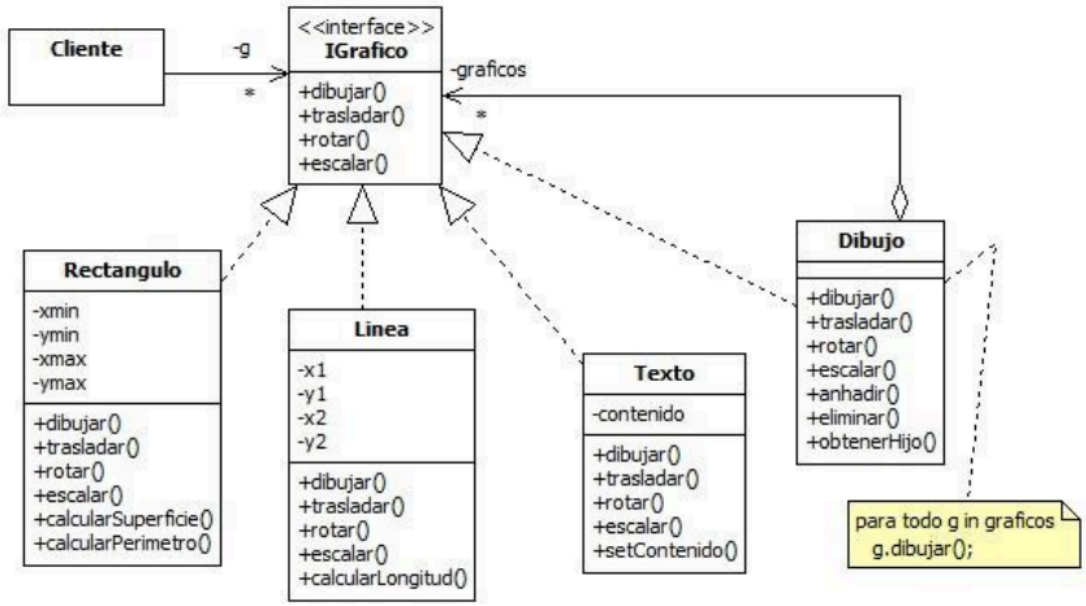
- **Bridge** conecta una abstracción con sus implementaciones, proporcionando una interfaz estable a los clientes y permitiendo que las implementaciones cambien de manera independiente. Se enfoca en **desacoplar** la **abstracción** de la **implementación**.

Se usan en diferente puntos del ciclo de vida: **Bridge** se utiliza en la **fase de diseño** para permitir que las abstracciones y las implementaciones cambien sin afectar a los clientes; y **Adapter**, por otro lado, se usa cuando ya tienes una interfaz existente que no es compatible con la que necesita el cliente, y por lo tanto, debes adaptar esas interfaces sin modificar el código original.

Composite

El patrón Composite organiza objetos en **estructuras jerárquicas de tipo árbol** para representar relaciones de "parte-todo", **jerarquías**. Este patrón permite tratar de manera **uniforme** tanto a los objetos **individuales** como a sus **composiciones** (es decir, los grupos de objetos). Esto es útil cuando se quiere tratar a los elementos de una jerarquía de forma similar, sin tener que hacer distinción entre los componentes individuales y las composiciones que los agrupan. Se usa:

- Para representar **jerarquías** de objetos con una relación parte-todo, donde tanto los objetos individuales como las composiciones deben tratarse de manera similar.
- Para **ignorar** las **diferencias** entre los objetos individuales y las composiciones (grupos de objetos) al interactuar con ellos, lo que simplifica el código y la manipulación de objetos en estructuras complejas.

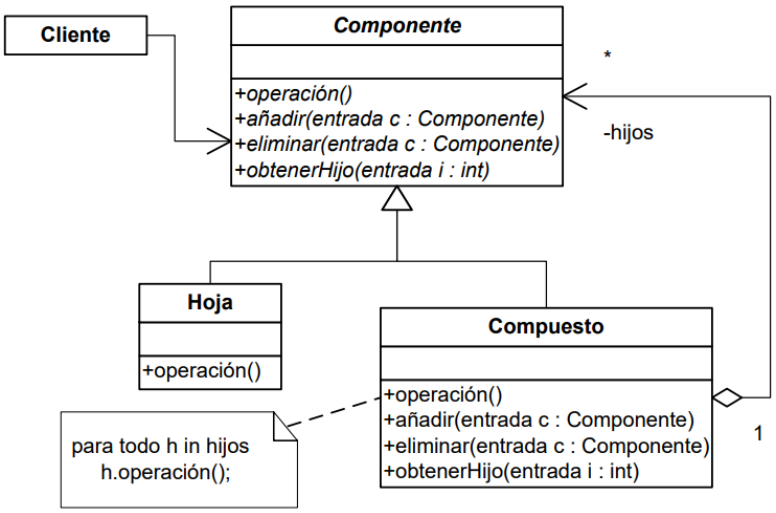


En la parte superior izquierda está la interfaz **IGrafico**, define los métodos generales que todos los gráficos deben ser capaces de ejecutar, como dibujar(), trasladar(), rotar() y escalar(); son comunes a todas las clases gráficas (simples o compuestas). Debajo de la interfaz IGrafico tenemos las clases **Rectángulo**, **Línea** y **Texto**, que son ejemplos de Hojas en el patrón Composite.

En el centro del diagrama, encontramos la clase **Dibujo**, que representa un Componente Compuesto (también implementa la interfaz IGrafico). Sin embargo, Dibujo puede contener otros objetos gráficos, que a su vez pueden ser Hojas o incluso otros Compositos. A través de estos métodos, un Dibujo puede contener varios elementos gráficos y delegar las operaciones a ellos, como se ve en el método dibujar(), que recursivamente llama a dibujar() en todos sus hijos.

- <u>Participantes</u>

- **Componente**: declara interfaz para los objetos de la agregación e implementa comportamiento predeterminado.
- **Hoja**: establece comportamiento de objetos primitivos (hojas).
- **Compuesto**: define comportamiento de componentes con hijos e implementa las operaciones de acceso a los hijos.
- **Cliente**: manipula objetos a través de la interfaz Componente.



- <u>Ventajas e inconvenientes</u>

El patrón permite que las composiciones sean **agregaciones recursivas**, lo que significa que un compuesto puede contener otros compuestos, creando estructuras complejas de forma sencilla. **Uniformiza** el **acceso** a los componentes: los clientes son simples porque no saben si tratan con hojas o compuestos. Facilita la **adición** de nuevos tipos de componentes sin modificar la estructura existente. Sin embargo, su diseño general dificulta restringir los componentes permitidos en un grupo.

- <u>Implementación</u>

Se pueden tomar las siguientes consideraciones clave:

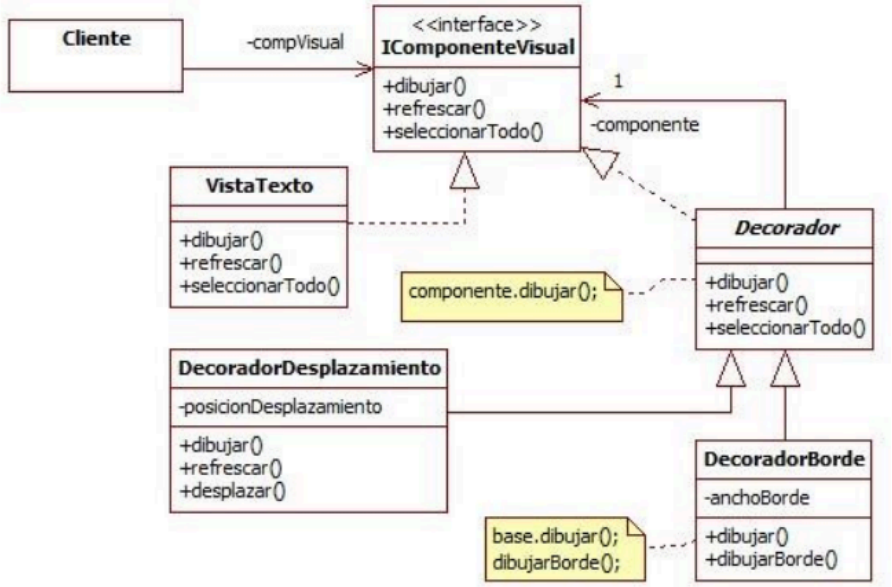
- **Referencias de hijos a padres**: Es recomendable mantener referencias de los hijos a sus padres dentro de la estructura, ya que esto **simplifica** el **recorrido** por la jerarquía de objetos.

- **Maximizar la interfaz de Componente**: Es importante definir tantas **operaciones comunes** entre las Hojas y los Compuestos como sea posible. Cuanto más similar sea la interfaz entre ambos tipos de objetos, más fácil será tratarlos de manera uniforme. Las operaciones que **no se aplican a los hijos** (como agregar o eliminar componentes) deben ser implementadas en la clase Componente para que se puedan manejar correctamente.

Decorator (Decorador)

El patrón **Decorator** (Decorador) es un patrón estructural que permite **asignar** o **retirar responsabilidades adicionales** a un objeto particular de forma **dinámica**, sin necesidad de modificar su clase original. En lugar de utilizar herencia para extender funcionalidades (lo cual es estático y rígido), el Decorator propone **envolver objetos** dentro de otros objetos que añaden comportamiento adicional (solución más flexible, ya que las responsabilidades pueden combinarse en tiempo de ejecución según las necesidades).

Se suele combinar con el patrón **Composite**. Y se usa: para otorgar o renovar responsabilidades a objetos individuales de manera dinámica y transparente, y cuando no resulta viable o práctico extender mediante herencia, o cuando se desea evitar jerarquías de clases excesivamente complejas.

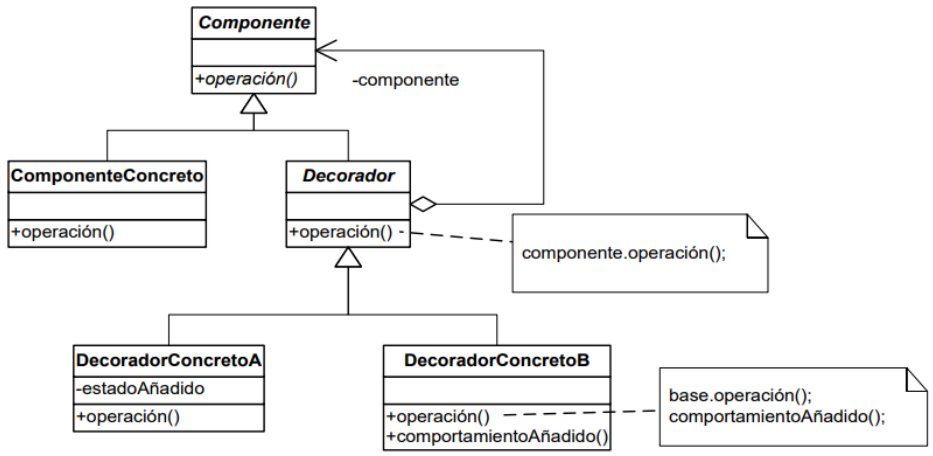


En la parte superior del diagrama aparece la interfaz **IComponenteVisual**, con las operaciones comunes, como dibujar(), refrescar() y seleccionarTodo(). A la izquierda se encuentra el **Cliente**, que trabaja exclusivamente con esta interfaz, sin conocer si está interactuando con un objeto base o con uno decorado. Debajo de la interfaz encontramos la clase **VistaTexto**, que representa el **ComponenteConcreto**. Esta clase implementa directamente la interfaz IComponenteVisual y proporciona el comportamiento básico, es decir, la

funcionalidad original sin ningún tipo de extensión. Es el objeto que será "envuelto" por los decoradores para añadir nuevas responsabilidades sin modificar su código. A la derecha del diagrama se encuentra la clase **Decorator**, que también implementa la interfaz y mantiene una **referencia** a un objeto de tipo **IComponenteVisual** (indicada como "componente"). A partir de este Decorator base, se derivan los **DecoradoresConcretos**, que son los que realmente añaden nuevas responsabilidades (DecoradorDesplazamiento, que introduce un nuevo atributo y amplía el comportamiento, y DecoradorBorde, que añade un atributo y redefine el método dibujar()).

- <u>Participantes</u>

- **Componente**: define la interfaz común que tanto los objetos originales como los decorados (a los que se pueden añadir responsabilidades) deben implementar.
- **ComponenteConcreto**: define el objeto base/original al que se le pueden añadir nuevas responsabilidades (decorar).
- **Decorador**: referencia a un Componente y se ajusta a su interfaz, actúa como una clase abstracta que también implementa la interfaz del componente y mantiene una referencia a este.
- **DecoradoresConcretos**: son los que realmente añaden nuevas funcionalidades o comportamientos al objeto (componente), ya sea antes o después de delegar la operación al componente original.



- <u>Ventajas e inconvenientes</u>

Destaca por ofrecer mayor **flexibilidad** que la herencia (estática), permitiendo combinar comportamientos de forma dinámica y **evitando** clases **sobrecargadas** de funciones en la parte superior de la jerarquía. Sin embargo, su uso suele dar lugar a muchos **objetos pequeños** y **parecidos** entre sí, lo que puede dificultar la comprensión del sistema.

- <u>Relación con otros patrones</u>

Se diferencia del **Adapter** en que no cambia la interfaz del objeto, sino que amplía sus responsabilidades. Además, puede considerarse una variante del patrón **Composite** desvirtuado con un único componente que añade responsabilidades, ya que ambos utilizan composición, aunque con objetivos distintos.

- <u>Composite vs Decorator</u>

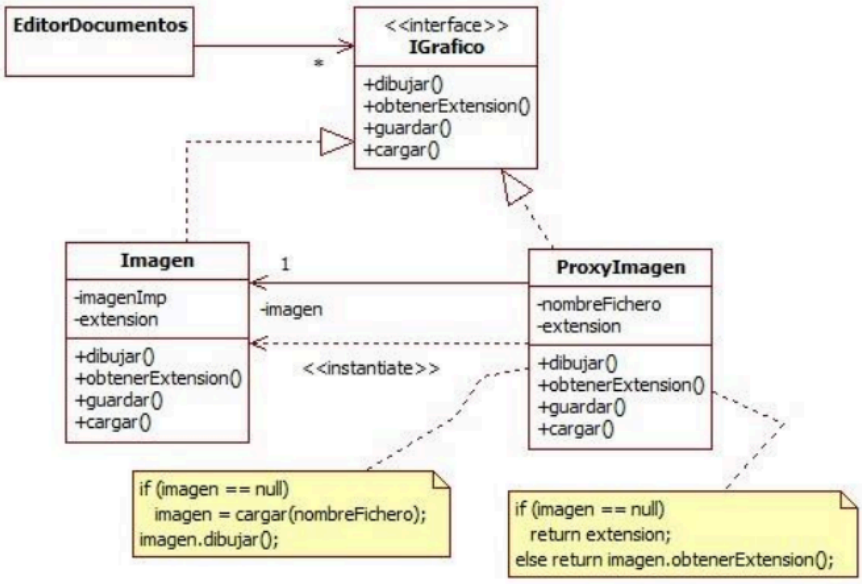
Ambos se basan en la agregación recursiva pero con objetivos diferentes: el **Decorator** se centra en añadir responsabilidades a objetos individuales de forma dinámica **sin crear subclases**, mientras que el **Composite** busca representar **jerarquías completas** y permitir uniformidad en el acceso a los elementos y a sus composiciones. Ambos propósitos pueden considerarse **complementarios** dentro del diseño de sistemas orientados a objetos.

Proxy (Apoderado)

El patrón **Proxy** es un patrón estructural cuyo objetivo es proporcionar un sustituto o representante de otro objeto para **controlar** el **acceso** a este. En lugar de interactuar directamente con el objeto real, el cliente se comunica con el Proxy, que actúa como **intermediario**. Una característica clave de este patrón es que **implementa la misma interfaz que el objeto representado**, de modo que el cliente no percibe ninguna diferencia en su uso. Esto permite introducir lógica adicional sin modificar el objeto real.

- <u>Aplicación</u>

- **Proxy remoto**: permite representar localmente un objeto que se encuentra en **otro espacio de direcciones**, ocultando los detalles de comunicación remota.
- **Proxy virtual**: se utiliza para optimizar recursos, creando objetos de alto coste únicamente cuando son realmente necesarios (por encargo).
- **Proxy de protección**: controla el acceso al objeto original, verificando permisos antes de permitir la operación.
- **Referencia inteligente**: sustituye a una referencia y añade operaciones adicionales, como contadores de referencias, gestión de memoria o registro de accesos.

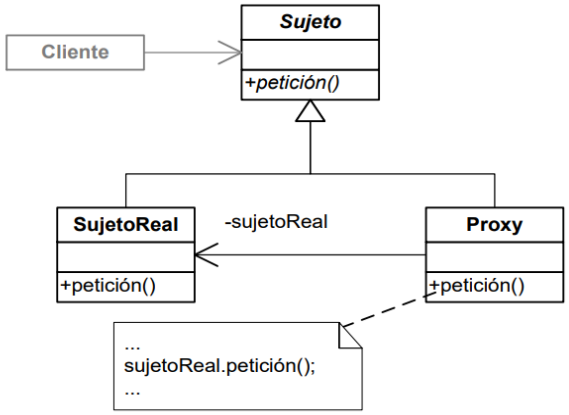


En la parte superior del diagrama aparece la interfaz **IGrafico**, que define las operaciones comunes como dibujar(), obtenerExtension(), guardar() y cargar(). A la izquierda se encuentra **EditorDocumentos**, que representa el cliente. Este componente solo conoce la interfaz IGrafico y no distingue si está interactuando con una imagen real o con un proxy. Debajo de la interfaz aparece la clase **Imagen**, que es el **ObjetoReal**, que implementa directamente IGrafico y contiene la funcionalidad completa. A la derecha se encuentra la clase **ProxyImagen**, que también implementa la interfaz IGrafico, pero en lugar de realizar directamente todas las operaciones, mantiene una **referencia** a un **objeto Imagen**.

- <u>Participantes</u>

- **Sujeto** (Subject): define la interfaz común que comparten tanto el Proxy como el SujetoReal.
- **SujetoReal** (RealSubject) es el objeto original que contiene la lógica principal del sistema.

El Proxy mantiene una **referencia** al **SujetoReal** y controla el acceso a este, pudiendo encargarse además de su creación, inicialización o destrucción. El cliente interactúa siempre con la interfaz Sujeto, sin saber si está trabajando directamente con el objeto real o con su proxy.



- <u>Ventajas e inconvenientes</u>

Permite introducir una **capa de indirección** adicional que aporta múltiples beneficios: el proxy remoto **oculta** que un objeto resida en un espacio de direcciones distinto, el proxy virtual **crea** un objeto por encargo, y el proxy de protección **regula** el **acceso** al objeto representado. Puede optimizar el uso de recursos (contadores de referencias) mediante **creación diferida de copias**.

- <u>Relación con otros patrones</u>

En relación con otros patrones, el Proxy se diferencia del Adapter en que este último **proporciona** una **interfaz diferente** para hacerla compatible con el objeto adaptado, mientras que el Proxy mantiene la **misma interfaz** que el objeto representado, pero puede rechazar peticiones soportadas por éste.

En comparación con el Decorator, ambos comparten una estructura similar y proporcionan una interfaz idéntica a la de un objeto original, pero su **propósito** es **distinto**: el Decorator **añade responsabilidades** o comportamientos de forma dinámica y recursivamente, mientras que el Proxy se centra en **proporcionar** un **sustituto** de un sujeto cuando no es conveniente acceder directamente al objeto real.